Ghép ảnh 2D từ video UAV biết rủi ro và thích ứng tài nguyên: một nghiên cứu thực nghiệm

Diệp Quang Sáng, Nguyễn Đức Nam, Đào Đặng Nguyên Khôi, Trương Quang Đăng Khoa
AI2002, Nhóm 09
Trường Đại học FPT, Thành phố Hồ Chí Minh

Abstract: Ghép ảnh tuần tự từ video UAV hỏng một cách âm thầm: một phép đăng ký sai bị tích luỹ vào bản đồ và mọi frame sau đều thừa hưởng sai số đó. Bộ đối sánh cổ điển rẻ nhưng dễ vỡ trên địa hình nghèo đặc trưng; bộ đối sánh học sâu bền nhưng quá chậm để chạy cho mọi frame. Chúng tôi nghiên cứu ba cơ chế trên một pipeline homography tích luỹ: bộ chỉ báo rủi ro rẻ ở cấp bước, bộ phân luồng đối sánh hai tầng, và ba chiến lược phục hồi, tất cả đều đo so với matcher tham chiếu SIFT ảnh gốc trên hai cửa sổ 45 bước của bộ dữ liệu NPU Drone-Map. Ba phát hiện chính: (i) chỉ báo đếm điểm *không* tách được bước sai (AUC 0,40 cho `matches`, 0,41 cho `inliers`, tức thấp hơn mức đoán mò), trong khi bằng chứng sau khi ghép thì tách được (AUC 0,77–0,86); (ii) chạy matcher học sâu cho *mọi* bước vừa chậm vừa tệ hơn chỉ dùng SIFT (60,4 s / 82,4 px so với 3,2 s / 75,3 px drift cuối), còn phân luồng ở τ_{ncc}=0,93 leo thang 16% số bước và đạt 28,5 px, nghĩa là điểm làm việc tốt nằm trong một khoảng hẹp, không đơn điệu theo ngưỡng; (iii) dưới cùng một ngưỡng kích hoạt, chiến lược tách mảnh giữ đủ 46/46 frame, còn bỏ frame và rematch keyframe chỉ giữ 14/46 do trigger dây chuyền. Kết luận bị giới hạn ở hai cửa sổ được chọn sẵn, một bộ dữ liệu, và thước đo sai số là đồng thuận với matcher tham chiếu.

Index Terms: Ghép ảnh UAV, dự báo lỗi đăng ký, phân luồng bộ đối sánh, ước lượng homography, trôi dạt tích luỹ, chiến lược phục hồi.

I. Giới thiệu

Ghép một chuyến bay UAV thành một ảnh 2D duy nhất là phương án rẻ hơn nhiều so với tái tạo quang trắc đầy đủ, khi mục tiêu chỉ là một bức ảnh hiện trạng cục bộ. Công thức chuẩn là tích luỹ: ghép frame i với frame $i-1$, ước lượng homography H_i, tích luỹ lại và hoà trộn các frame đã warp. Kiểu hỏng cũng chuẩn không kém: trên địa hình nghèo vân hoặc lặp vân, ước lượng từng cặp suy biến, và vì mỗi bước đều điều kiện hoá theo bước trước, một H_i sai sẽ kéo lệch toàn bộ các frame phía sau (tích luỹ drift).

Văn liệu đưa ra hai thái cực. Pipeline thủ công (SIFT/ORB với RANSAC [1], [2], [3]) đủ nhanh để chạy CPU tại hiện trường nhưng vỡ ở frame khó; matcher học sâu (SuperPoint+LightGlue [4], [5], họ LoFTR [6], [7]) xử lý được các frame đó nhưng tốn hơn một đến hai bậc độ lớn, điều này quan trọng khi cả chuyến bay phải xử lý trên laptop hay trong một phiên Colab. Các hệ thống tích luỹ hiện có phần lớn nhận hoặc loại frame bằng một ngưỡng kinh nghiệm cố định [9], [10] chứ không quyết định *frame này đáng bao nhiêu tính toán*, và các hành vi phục hồi xuất hiện rải rác trong từng công trình (bỏ frame, rematch keyframe cũ, đóng mảnh con) chưa từng được so sánh dưới cùng một ngưỡng kích hoạt trên cùng một pipeline.

Bài này báo cáo một nghiên cứu thực nghiệm trả lời bốn câu hỏi trên một pipeline tích luỹ:

- **RQ1 (dự báo lỗi).** Chỉ báo rẻ nào ở cấp bước thật sự tách được bước ghép sai khỏi bước ghép tốt?
- **RQ2 (phân luồng).** Chính sách hai tầng (mặc định dùng matcher cổ điển, chỉ gọi matcher học sâu ở bước bị gắn cờ) mua được gì về chi phí và về chất lượng bản đồ, so với luôn-cổ-điển và luôn-học-sâu?
- **RQ3 (phục hồi).** Dưới cùng một ngưỡng, bỏ frame, rematch frame đã đặt và tách mảnh con khác nhau thế nào về số frame giữ được và về drift?

Đóng góp của chúng tôi chính là các số đo, trong đó có hai kết quả phủ định mà chúng tôi cho là phần dễ chuyển giao nhất: số inlier phản tác dụng ở cấp bước, và gọi matcher mạnh cho mọi bước làm bản đồ *tệ hơn* matcher rẻ trên cửa sổ đã đo.

II. Công trình liên quan

Ghép ảnh tích luỹ. Các pipeline tuần tự hướng CPU giới hạn số đặc trưng hoặc dùng ngưỡng inlier nhẹ để giữ chi phí đăng ký từng cặp ở mức chấp nhận được [9]; chọn keyframe và tối ưu sai số chiếu lại có trọng số được dùng để hãm drift [10]; chọn seamline và phép warp xử lý phần hoà trộn [12]; pipeline nông nghiệp dùng dòng quang học và phân cụm frame thành mảnh con [11]. Tất cả đều có luật nhận/loại dạng kinh nghiệm, nhưng không có quyết định biết đến chi phí.

Độ tin cậy của đối sánh và dự báo lỗi. Trong nhận diện vị trí bằng thị giác, số inlier và tỉ lệ inlier tách được ca khớp thành công khỏi ca thất bại, với dương tính giả ở vùng vân lặp [15], [16]. Ở mức homography, các benchmark quy mô lớn cho thấy sai số chiếu lại và số lượng đặc trưng tương quan yếu với sai số hình học cuối cùng [17], [18]. Ước lượng độ bất định đã được thêm vào ước lượng homography học sâu cho định vị vệ tinh–nhiệt [20], và các kiểm tra nhất quán giữa mô hình như VSAC chấm điểm chất lượng inlier một cách độc lập [19]. Các công trình này là động lực cho bộ chỉ báo rẻ, nhưng vẫn để mở câu hỏi chỉ báo nào dự báo sai số *cấp bước* trong một chuỗi ghép.

Cổ điển so với học sâu. SIFT+LightGlue được chứng minh cứu được vùng nghèo vân và xoay góc lớn trên UAV [13];

so sánh có hệ thống trên ảnh di động và ảnh drone cho thấy phương pháp học được chính xác hơn nhưng luôn chậm hơn [14]. Các matcher học nhẹ (XFeat [8]), matcher bán dày có chọn token (Efficient LoFTR [7]) và thiết kế attention riêng cho UAV [21] làm giảm chứ không xoá bỏ chi phí đó. Bộ phân luồng của chúng tôi là phía "cổ điển trước" của đánh đổi này, và được đánh giá ở mức bản đồ chứ không phải mức từng cặp ảnh.

III. Phương pháp

A. Pipeline

Frame được đọc theo thứ tự thời gian. Với bước $(i-1, i)$, pipeline trích đặc trưng, đối sánh, ước lượng H_i bằng RANSAC rồi tích luỹ $H_{i\to\text{root}}$. Tầng rẻ là SIFT + brute-force với tỉ lệ Lowe, ở `max_side`=640 và `max_keypoints`=2000. Tầng mạnh là SuperPoint + LightGlue trên đúng ảnh 640 px đó; một biến thể tuỳ chọn chạy lại các bước bị gắn cờ ở độ phân giải gốc với cùng matcher. Cài đặt: OpenCV 5.0.0, PyTorch 2.14.1 (chỉ CPU), NumPy 2.4.6, ngưỡng RANSAC 3 px, seed 0.

B. Sai số cấp bước so với matcher tham chiếu

Các gói dữ liệu này không có ground truth ngoài thực địa, nên sai số được định nghĩa là mức bất đồng với một ước lượng tham chiếu. Với mỗi bước, matcher tham chiếu (SIFT ảnh gốc, 4000 đặc trưng, cùng RANSAC) cho H_i^{ref}; bước có nghiệm tham chiếu dưới 100 inlier bị coi là không hợp lệ. Sai số bước là độ lệch trung bình của năm điểm mốc (bốn góc và tâm frame nguồn, trong hệ toạ độ ảnh gốc) giữa homography của pipeline và H_i^{ref}:

$$e_i = \frac{1}{5}\sum_{k=1}^{5} \left\| H_i x_k - H_i^{\text{ref}} x_k \right\|_2 . \tag{1}$$

Một bước là *sai* khi $e_i > 2$ px, tức mức nền nhiễu đo được của chính matcher tham chiếu trên dữ liệu này. Drift cả chuỗi được báo cáo riêng, là hiệu tích luỹ giữa chuỗi của pipeline và chuỗi tham chiếu, đơn vị pixel.

C. Chỉ báo và ngưỡng kích hoạt

Tám chỉ báo rẻ được ghi cho mỗi bước: `matches`, `inliers`, `inlier_ratio`, `rmse_px` (RMSE nội tại của RANSAC), `hull_area_ratio` (độ phân tán không gian của inlier), `overlap_ratio` (giao khung ảnh sau khi chiếu), `photo_ncc` (tương quan chéo chuẩn hoá của cặp ảnh sau khi ghép) và `photo_bad_frac` (tỉ lệ điểm ảnh lệch quá 25 mức xám sau khi ghép). Mỗi chỉ báo có *chiều nghi ngờ* được cố định *trước* khi đánh giá (ví dụ `photo_bad_frac` cao là đáng ngờ, `photo_ncc` thấp là đáng ngờ), nên không có chiều nào được chọn sau khi nhìn nhãn. Một bước bị gắn cờ khi `photo_ncc` $< \tau$, hoặc `inlier_ratio` $< 0{,}30$, hoặc không sinh được homography. τ được chọn bằng luật chứ không bằng mắt: giá trị lớn nhất sao cho cả hai cửa sổ đánh giá vẫn nằm trong ngân sách leo thang 25%.

D. Các chiến lược phục hồi

Bốn chiến lược chạy trên cùng pipeline và cùng ngưỡng: *không cứu* (vẫn đăng ký), *bỏ frame*, *neo* (rematch với frame đã đặt gần nhất thay vì với frame liền trước) và *tách mảnh* (đóng mảnh con hiện tại và mở mảnh mới tại frame đó). Phục hồi được đo bằng số frame đặt được, số mảnh con, số bước ghép được ghi, và drift nội bộ lớn nhất của mỗi chuỗi.

IV. Thiết lập thực nghiệm

Dữ liệu. Các gói keyframe công khai của NPU Drone-Map [22] (RGB 1920×1080 đã khử méo, khoảng 400 frame mỗi gói). Hai gói được dùng: `phantom3-village-kfs` (cửa sổ khó = frame 80–125, nơi SIFT bất đồng với tham chiếu 5–11 px) và `phantom3-grass-kfs` (frame 0–45, nơi Efficient LoFTR bất đồng 5–313 px). Cửa sổ được chọn sau khi quét 160 bước cho mỗi gói; 20 frame đầu bị loại vì cả bảy cấu hình matcher ghép đủ 20 frame đó mà không dừng lại, nên không có ca hỏng nào để dự báo. Mỗi cửa sổ là 46 frame / 45 bước. Artifact gồm SHA-256 của cả 92 ảnh đầu vào, manifest môi trường và bản ghi từng bước; toàn bộ notebook chạy trên CPU trong 507 s. Ảnh ghép được sinh bởi `reports/paper/make_mosaics.py`, script phát lại các homography đã lưu mà không chạy lại matcher, và tự kiểm tra rằng drift của mọi chuỗi dựng lại bằng đúng giá trị trong bảng.

Thước đo. Với RQ1: Spearman ρ giữa chỉ báo và sai số bước, và ROC-AUC cho nhãn sai/tốt, dùng chiều nghi ngờ đã khai báo trước; AUC chỉ báo cáo khi cả hai lớp đều khác rỗng. Với RQ2: thời gian matcher cộng gộp (giây, chỉ tầng matcher), tỉ lệ leo thang, sai số bước trung bình và drift cuối. Với RQ3: số frame đặt được, số mảnh con, số bước ghép được ghi, và drift nội bộ lớn nhất.

V. Kết quả

A. RQ1: chỉ báo nào dự báo được bước sai

Bảng I là cửa sổ village/SIFT, nơi 21 trên 45 bước là sai. Hai chỉ báo đếm điểm là tệ nhất trong nhóm: cả hai đều có chiều khai báo (càng ít điểm khớp hoặc ít inlier càng đáng ngờ) bị dữ liệu đảo ngược, cho AUC 0,402 (`matches`) và 0,409 (`inliers`), nghĩa là các bước có *nhiều* điểm khớp hơn lại hơi dễ sai hơn. Bằng chứng sau khi ghép áp đảo: bất đồng quang học sau warp (0,855), RMSE nội tại (0,839), độ chồng lấn (0,788) và NCC (0,770). Thứ tự này giữ nguyên với SuperPoint+LightGlue trên cùng cửa sổ (27/45 bước sai; `photo_ncc` 0,798, `photo_bad_frac` 0,757, `matches` 0,436) và trên grass/SuperPoint+LightGlue (6/38 bước sai; `photo_ncc` 0,865, `matches` 0,615). Hai cửa sổ không có AUC xác định: grass/SIFT có 0 bước sai (0/38 bước hợp lệ), còn Efficient LoFTR trên village sai 45/45 bước, đây là ca chẩn đoán chứ không phải tín hiệu học được.

B. RQ2: chi phí và chất lượng của phân luồng

Bảng II và Hình 3(b) cho thấy đánh đổi trên village. Luôn-học-sâu là tệ nhất trong ba chính sách ở *cả hai* trục: tốn gấp 18,7 lần thời gian matcher của chỉ-SIFT và kết thúc xa chuỗi tham chiếu hơn 9% (82,4 px so với 75,3 px). Phân luồng ở

TABLE I

RQ1, cửa sổ `village`/SIFT+BF: 45 bước, 21 bước gắn nhãn sai. Chiều là dấu nghi ngờ đã khai báo trước; AUC tính theo dấu đó, còn Spearman ρ tính trên giá trị thô của chỉ báo, nên AUC dưới 0,5 nghĩa là chỉ báo bị đảo chiều so với khai báo trong cửa sổ này.

Chỉ báo	Chiều	Spearman ρ	AUC
`photo_bad_frac`	↑	+0,759	0,855
`rmse_px`	↑	+0,788	0,839
`overlap_ratio`	↓	−0,634	0,788
`photo_ncc`	↓	−0,636	0,770
`inlier_ratio`	↓	−0,389	0,726
`hull_area_ratio`	↓	+0,005	0,571
`inliers`	↓	+0,239	**0,409**
`matches`	↓	+0,251	**0,402**

TABLE II

RQ2, cửa sổ `village` (45 bước). "Mạnh" = số bước gửi sang SuperPoint+LightGlue; thời gian là của riêng tầng matcher.

Chính sách	Mạnh	Thời gian (s)	e_i TB (px)	Drift (px)
Chỉ SIFT	0/45	3,2	2,51	75,3
Chỉ SP+LG	45/45	60,4	2,67	82,4
Phân luồng τ=0,93	7/45	13,3	2,42	**28,5**
Phân luồng + ảnh gốc	7/45	73,6	**1,95**	35,8

τ_{ncc}=0,93 leo thang 7 trên 45 bước (15,6%, nằm trong ngân sách 25%) và giảm drift cuối từ 75,3 px xuống 28,5 px, tức thấp hơn 62%, với chi phí matcher gấp 4,1 lần chỉ-SIFT. Leo thang đúng bảy bước đó ở ảnh gốc làm sai số cấp bước tốt hơn nữa (1,49 px so với 4,51 px trên các bước bị gắn cờ) nhưng lại kết thúc với drift chuỗi lớn hơn (35,8 px); xem thảo luận ở Mục VI.

Trên grass, bức tranh khác: SIFT mới là matcher chính xác ở đó (0 bước sai so với tham chiếu; drift cuối 14,7 px trong 2,4 s), còn SuperPoint+LightGlue tốn 49,0 s và drift 38,4 px. Do đó luật ngân sách chọn τ=0,96 và không leo thang bước nào, tức bộ phân luồng từ chối tiêu tính toán đúng chỗ. Hình 1 và Hình 2 cho thấy bản đồ mà các chính sách này tạo ra.

C. RQ3: phục hồi dưới cùng một ngưỡng

Bảng III báo cáo cả hai cửa sổ. Trên village (7 trigger ban đầu), *tách mảnh* đặt đủ 46 frame thành 8 mảnh con, nối được 38 trên 45 bước với drift nội bộ lớn nhất 21,7 px; *bỏ frame* và *neo* chỉ giữ 14/46 frame (13 bước nối được, cùng mức trần 21,7 px), vì bỏ một frame làm tăng khoảng trống chồng lấn của cặp kế tiếp và ngưỡng lại kích hoạt (7 trigger ban đầu nở thành 32). *neo* cho kết quả y hệt *bỏ frame* ở đây: khi mọi ứng viên còn lại đều đã bị loại hoặc chưa được đặt, không có frame neo nào an toàn. Hình 5 cho thấy chiến lược tách mảnh thực sự tạo ra cái gì.

Trên grass với Efficient LoFTR, 44 trong 45 bước bị gắn cờ. *bỏ frame* và *neo* đặt được đúng một frame (bản đồ không tồn tại); *tách mảnh* giữ đủ 46 frame nhưng suy biến thành 45 mảnh một frame với đúng một bước nối được. Drift thấp ở đó là hệ quả của việc chặt vụn, không phải bằng chứng của một bản đồ được cứu.



Fig. 1. Ảnh ghép cửa sổ `village` (46 frame, bay thẳng), dựng lại từ homography đã lưu: (a) chỉ SIFT, (b) phân luồng ở τ_{ncc}=0,93. Đường viền ngoài là biên frame đã warp; khác biệt drift (75,3 px so với 28,5 px) là số đo, không phải đọc bằng mắt trên hình.

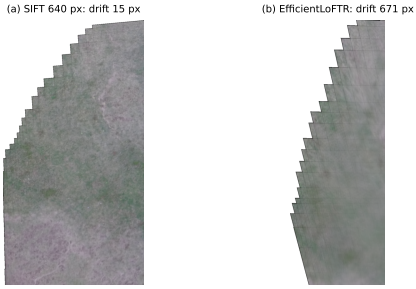


Fig. 2. Cùng 46 frame của `grass`, hai matcher, không phục hồi: (a) SIFT 640 px, (b) Efficient LoFTR. Vệt răng cưa bên phải là drift tích luỹ 671,3 px so với chuỗi tham chiếu, trên địa hình mà matcher học sâu vẫn báo điểm khớp dày và RMSE thấp nhưng sai hình học.

VI. Thảo luận

Rẻ không có nghĩa là có ích. Số điểm khớp và số inlier, những chỉ báo mà hầu hết luật nhận/loại dùng, không mang tín hiệu dùng được ở cấp bước trong cửa sổ của chúng tôi, và dữ liệu còn đảo ngược chiều đã khai báo (càng ít điểm khớp càng bị coi là đáng ngờ): matcher sinh nhiều điểm khớp vẫn có thể đặt chúng nhất quán với một hình học sai. Các chỉ báo tính *sau* khi đăng ký (tương phản quang học, sai số nội tại, độ chồng lấn sau chiếu) mới mang tín hiệu. Hệ quả thực tế: chúng không thể đánh giá trước khi matching, nên ngưỡng của chúng tôi là một cổng hậu kiểm trên kết quả của matcher rẻ, không phải bộ dự báo trước matching.

Tiêu nhiều tính toán hơn không đơn điệu theo chất lượng. Luôn-học-sâu kết thúc tệ hơn chỉ-SIFT trên village, và việc leo thang ở ảnh gốc cải thiện sai số từng bước nhưng không chuyển thành drift chuỗi thấp hơn. Cơ chế khả dĩ: bản đồ tích luỹ là sản phẩm của cả chuỗi, không phải của sai số trung bình từng bước; một H_i "tốt hơn" cục bộ làm thay đổi hình học mà các frame sau phải bù lại. Chúng tôi coi đây là giả thuyết; muốn kiểm thì phải thay đổi tập bước leo thang độc lập với ngưỡng.

Chọn ngưỡng mới là phần khó. Hình 4 cho thấy đường quét có dạng chữ U: 15,6% leo thang là điểm tốt nhất đo được, còn leo thang một nửa số bước (53%) lại tệ hơn cả không leo thang bước nào. Luật ngân sách ("τ lớn nhất sao cho tỉ lệ leo thang dưới 25%") cho 0,93/0,96 trên hai cửa sổ, nhưng một luật giả định tính đơn điệu trên một khoảng mà thước đo

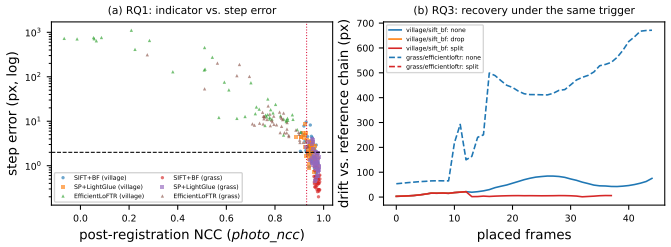


Fig. 3. (a) Sai số bước theo chỉ báo NCC sau ghép trên cả hai cửa sổ và ba matcher; đường gạch ngang = ranh giới 2 px, đường chấm = ngưỡng được chọn. (b) Drift so với chuỗi tham chiếu của các chiến lược RQ3 dưới cùng ngưỡng. Tách mảnh đặt lại hệ toạ độ ở mỗi mảnh, nên đường của nó là từng đoạn, không phải một chuỗi liền.

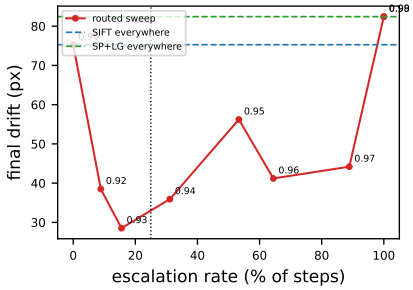


Fig. 4. Quét ngưỡng RQ2 trên `village`: tỉ lệ leo thang so với drift cuối. Hai đường ngang là hai chính sách cố định; đường chấm là ngân sách leo thang 25%. Drift không đơn điệu theo τ, nên điểm làm việc phải chọn theo ngân sách chứ không theo tâm lý "ngưỡng cao hơn là an toàn hơn".

không đơn điệu thì chỉ đúng trong phạm vi những cửa sổ đã dùng để khớp nó.

Đánh đổi của phục hồi là frame so với tính liên thông. Tách mảnh bảo toàn số frame nhưng làm vụn bản đồ (8 mảnh cho 46 frame, 45 mảnh trong ca suy biến); bỏ frame/neo giữ được tính liên tục của phần sống sót nhưng bị dây chuyền dưới một ngưỡng bảo thủ, mất 70% số frame ở đây. Bên nào tốt hơn phụ thuộc vào việc dùng bước sau cần độ phủ hay cần một mặt phẳng hình học duy nhất, câu hỏi mà số liệu của chúng tôi chưa trả lời được, và đó là lý do nên xem ảnh ghép của từng chiến lược trước khi kết luận.

VII. Giới hạn

Thước đo sai số. Mọi sai số là bất đồng với chuỗi tham chiếu SIFT ảnh gốc, và chính chuỗi đó cũng tích luỹ sai số; chưa có phần nào được kiểm chứng với hình học thực địa (chưa đối chiếu GCP/ảnh trực giao trong bản nháp này), nên không được đọc các giá trị drift tuyệt đối như độ chính xác định vị.

Mẫu. Hai trong 14 gói công khai của NPU Drone-Map, hai cửa sổ được chọn *vì* chúng khó. Tỉ lệ bước sai (21/45, 6/38, 45/45) là đặc tính của cách chọn đó, không phải kỳ vọng khi triển khai. Chưa có sequence giữ riêng, và ngưỡng lẫn ngân sách chưa được thử lại trên các gói còn lại.

Chọn mẫu và đa giả thuyết. Chỉ báo, chiều, mốc 2 px và τ đều được chọn khi đã nhìn vào chính các cửa sổ này; chiều được cố định trước để loại lỗ hổng rõ nhất, nhưng kết quả



Fig. 5. Chiến lược tách mảnh trên `village`: vẫn 46 frame đó, cắt tại 7 bước bị gắn cờ thành 8 mảnh, mỗi mảnh một hệ toạ độ riêng. Frame không mất, nhưng đầu ra là 8 mặt phẳng rời rạc, trong đó bốn mảnh (3, 5, 6 và 8) chỉ có một frame.

TABLE III
RQ3: Các chiến lược phục hồi ở ngưỡng mà RQ2 đã chọn (village τ=0,93, grass τ=0,96). Drift là mức lớn nhất trong nội bộ một chuỗi; tách mảnh đặt lại hệ toạ độ cho từng mảnh, và n/a nghĩa là không đặt được frame nào nên không có gì để đo.

Cửa sổ / matcher	Chiến lược	Frame giữ	Mảnh con	Drift (px)
village / SIFT	không cứu	46/46	1	84,6
village / SIFT	bỏ frame	14/46	1	21,7
village / SIFT	neo	14/46	1	21,7
village / SIFT	tách mảnh	46/46	8	21,7
grass / ELoFTR	không cứu	46/46	1	671,3
grass / ELoFTR	bỏ frame	1/46	1	n/a
grass / ELoFTR	tách mảnh	46/46	45	2,7

vẫn là khám phá chứ chưa phải xác nhận. 45 bước cũng phụ thuộc thời gian, không phải 45 quan sát độc lập.

Kế toán chi phí. Cột thời gian chỉ đo tầng matcher. RANSAC, warp, các phép kiểm quang học và quyết định rủi ro đều không tính, nên bản nháp này không ủng hộ bất kỳ tuyên bố nào về độ trễ end-to-end; chi phí phân luồng thực tế cao hơn 13,3 s và chi phí của luôn-học-sâu cũng còn phần hậu xử lý chưa tính.

Tích hợp. RQ3 chạy một matcher cố định cho mỗi cửa sổ và không dùng các tuyến phân luồng từ RQ2; một lần chạy end-to-end "rẻ → leo thang → phục hồi", ảnh ghép của nó, và thước đo liên thông giữa các mảnh vẫn còn thiếu. Toàn bộ thời gian đo trên một máy chỉ có CPU.

VIII. Kết luận

Trên một pipeline homography tích luỹ cho video UAV, các chỉ báo rẻ dự báo được rủi ro cấp bước không phải là số điểm khớp mà các luật nhận/loại thường dùng, mà là bằng chứng sau khi ghép; một bộ phân luồng cổ-điển-trước với ngưỡng chọn theo ngân sách đã giảm drift cuối 62% với chi phí gấp 4,1 lần matcher rẻ, trong khi gọi matcher học sâu cho mọi bước thì vừa chậm vừa tệ hơn; và dưới cùng một ngưỡng, tách mảnh giữ được toàn bộ frame trong khi bỏ frame/neo bị dây chuyền xuống còn 14/46. Các bước tiếp theo, theo thứ tự: chạy lại nguyên notebook trên các gói giữ riêng, nối phân luồng với phục hồi thành một lần chạy end-to-end có đo thời

gian toàn pipeline, kiểm chứng với hình học thực địa, và xem ảnh ghép của từng chiến lược.

References

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.

[2] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2011, pp. 2564–2571.

[3] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[4] D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2018, pp. 224–236.

[5] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[6] J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, "LoFTR: Detector-free local feature matching with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 8922–8931.

[7] Q. Wang *et al.*, "Efficient LoFTR: Semi-dense local feature matching with sparse-like speed," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[8] P. Potlapalli, P. Lindenberger, and M. Pollefeys, "XFeat: Accelerated features for lightweight image matching," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[9] D. Hwang *et al.*, "Real-time 2D orthomosaic mapping using CPU-based incremental homography matrix estimation with UAV video," *Appl. Sci.*, vol. 16, no. 4, p. 2133, 2026.

[10] X. Li *et al.*, "A real-time incremental video mosaic framework for UAV remote sensing," *Remote Sens.*, vol. 15, no. 8, p. 2127, 2023.

[11] M. Kharismawati and D. Kazic, "DroneZaic: A robust end-to-end pipeline for mosaicking freely flown aerial video of agricultural fields," *Plant Phenome J.*, vol. 8, no. 1, p. e70033, 2025.

[12] J. Chen *et al.*, "UAV video stitching based on optimal seamline and half-projective warp," *Remote Sens.*, vol. 14, no. 5, p. 1068, 2022.

[13] J. Kim and S. Kim, "Robust UAV image mosaicking using SIFT and LightGlue," *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. XLVIII-2-W11-2025, pp. 169–173, 2025.

[14] S. Gaisbauer, P. Gyawali, Q. Zhang, O. Wysocki, and B. Jutzi, "To glue or not to glue? Classical vs learned image matching for mobile mapping cameras to textured semantic 3D building models," *ISPRS Ann. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. X-1/W2-2025, 2025.

[15] M. Zaffar *et al.*, "On the estimation of image-matching uncertainty in visual place recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[16] C. Sferrazza *et al.*, "To match or not to match: Evaluating keypoint matching confidence for visual place recognition," in *Proc. CVPRW*, 2024.

[17] D. Barath *et al.*, "A large-scale homography benchmark," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[18] H. Yuan, D. Peng, and M. Dong, "Comparative analysis of color space transformations on feature-based image registration," *J. Imaging*, vol. 10, no. 5, p. 105, 2024.

[19] M. Ivashechkin, D. Barath, and J. Matas, "VSAC: Efficient and accurate estimator for H and F," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021.

[20] J. Xiao and G. Loianno, "UASTHN: Uncertainty-aware deep homography estimation for UAV satellite–thermal geo-localization," arXiv:2502.01035, 2025.

[21] H. Zhang, H. He, Y. Zhou, and F. Lei, "Efficient image matching for UAV visual navigation via DALGlue," *Sci. Rep.*, vol. 15, p. 37684, 2025.

[22] S. Bu, Y. Zhao, G. Wan, and Z. Liu, "Map2DFusion: Real-time incremental UAV image mosaicing based on monocular SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2016.